

Razvoj web aplikacija - Predavanje 9

State management, UX stanja i forme

mag.ing. Josip Torić (SIT UNIZD)

07.12.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 State management: zašto i kad koristiti store
- 2 Pinia store za poslovne entitete
- 3 Lista s loading/error/empty stanjima
- 4 Composables: ponovno iskoristiva reaktivna logika
- 5 Forme s klijentskom i serverskom validacijom
- 6 Registracije i status badgevi
- 7 Reusable komponente (tablica, forma)
- 8 Commitovi za ovo predavanje

Na kraju predavanja: club korisnik može vidjeti svoje natjecatelje, dodati novog i prijaviti ga na natjecanje.

Ishodi učenja

Nakon ovog predavanja student može:

- kreirati Pinia store za CRUD operacije nad entitetom
- implementirati loading, error i empty stanja u viewu
- izdvojiti reaktivnu logiku u composable funkciju
- napraviti formu s v-model bindingom i klijentskom validacijom
- prikazati serverske greške u formi za statute 422 i 409
- izdvojiti ponavljajući UI u reusable komponentu s tipiziranim propsima

Kad koristiti store, a kad lokalni state

	Lokalni state (<code>ref()</code>)	Pinia store
Scope	Jedna komponenta	Cijela aplikacija
Primjer	Form inputi, toggle	Lista natjecatelja, auth
Persistencija	Do unmounta	Do refresha (ili <code>localStorage</code>)
Dijeljenje	Props/emit	Import u bilo kojoj komponenti

Pravilo: ako dva ili više viewova trebaju iste podatke, stavi u store. Ako je podatak relevantan samo unutar jedne komponente, drži ga lokalno.

State management: problem koji rješava

Bez centralnog storea, dijeljenje podataka među komponentama zahtijeva **prop drilling**:

App → Layout → Sidebar → UserInfo

Svaka komponenta u lancu mora primiti i proslijediti prop koji koristi samo zadnja. Ovo je:

- Nepregledno - 5 razina prosljeđivanja za jedan podatak
- Krto - promjena na jednoj razini razbija lanac
- Teško za maintain - ne znaš odakle podatak dolazi

Store rješava ovo: bilo koja komponenta može direktno čitati i pisati dijeljeni state bez posrednika.

Pinia vs Vuex

	Vuex (legacy)	Pinia
API	Options: state, mutations, actions, getters	Composition: ref, computed, function
Mutacije	Obavezne (sinkrone)	Nema - direktno mijenjaj state
TypeScript	Loša podrška	Odlična
DevTools	Da	Da (bolje)
Moduli	Kompleksni namespace	Svaki store je nezavisan

Pinia je službeni nasljednik Vuexa. Ako naiđete na tutoriale s Vuexom, koncepti su isti, ali API je drugačiji.

Store pattern: state, actions, getters

Svaki Pinia store ima tri koncepta:

Koncept	U Piniji	Analogija (backend)
State	<code>ref()</code>	Podatak u bazi
Getters	<code>computed()</code>	Izvedeni podatak (SQL VIEW)
Actions	Funkcije	Service metode

- **State** drži sirove podatke (lista natjecatelja, loading flag)
- **Getters** deriviraju podatke iz statea (filtrirani natjecatelji, ukupan broj)
- **Actions** mijenjaju state (dohvat s API-ja, dodavanje, brisanje)

Optimistički vs pesimistički update

Pesimistički

- Pošalji request → čekaj odgovor → ažuriraj UI
- Korisnik vidi spinner
- Sigurniji - nema rollbacka
- Sporiji UX

Optimistički

- Ažuriraj UI odmah → pošalji request → rollback ako ne uspije
- Korisnik vidi instant promjenu
- Kompleksniji - treba rollback logika
- Brži UX

Za ovaj projekt koristimo pesimistički pristup - jednostavniji je i dovoljno brz. Optimistički update se koristi za akcije poput lajkanja objave ili micanja itema iz liste.

Pinia store za natjecatelje

```
// src/stores/lifters.ts
import { defineStore } from 'pinia'
import { ref } from 'vue'
import api from '@api/client'
import type { Lifter } from '@types'

export const useLifterStore = defineStore('lifters', () => {
  const lifters = ref<Lifter[]>([])
  const loading = ref(false)
  const error = ref<string | null>(null)

  async function fetchLifters(clubId: number) {
    loading.value = true
    error.value = null
    try {
      const { data } = await api.get<Lifter[]>(
        `/clubs/${clubId}/lifters`
      )
      lifters.value = data
    } catch {
      error.value = 'Greska pri dohvat natjecatelja'
    } finally {
      loading.value = false
    }
  }

  return { lifters, loading, error, fetchLifters }
})
```

Zamka: destrukuiranje storea gubi reaktivnost

```
const store = useLifterStore()
const { lifters } = store // GUBI reaktivnost!

import { storeToRefs } from 'pinia'
const { lifters, loading } = storeToRefs(store) // ispravno
const { fetchLifters } = store // funkcije smiju direktno
```

Destrukiranje kopira trenutnu vrijednost i template se prestaje ažurirati kad se store promijeni. `storeToRefs` zadržava reaktivne reference, a akcije se smiju uzeti direktno jer su obične funkcije.

Napomena: AI alati često generiraju upravo ovu grešku - provjerite svako destrukuiranje storea koje vam alat predloži.

Tri stanja svakog viewa

Svaki view koji dohvaća podatke s API-ja mora hendlati tri situacije:

Stanje	Uvjet	Što prikazujemo
Loading	<code>loading === true</code>	Spinner ili skeleton
Error	<code>error !== null</code>	Poruka greške s retry gumbom
Empty	<code>items.length === 0</code>	Poruka i CTA (npr. „Dodaj prvog natjecatelja”)

Ako pokažete samo podatke bez ovih stanja, korisnik ne zna učitava li se nešto, je li greška ili stvarno nema podataka. To je loš UX.

Skeleton loading vs spinner

Spinner

- Generički indikator učitavanja
- Ne govori korisniku što će vidjeti
- Jednostavan za implementaciju
- Može uzrokovati layout shift

Skeleton screen

- Sivi blokovi na mjestu budućeg sadržaja
- Korisnik zna kakav layout očekivati
- Perceptualno brže (manje čekanja)
- Zahtijeva više koda

Za studentski projekt spinner je dovoljno dobar. Skeleton je standard u produkcijskim aplikacijama - Facebook, YouTube, LinkedIn ga koriste svugdje.

Empty state: zašto je bitan

Empty state je ono što korisnik vidi kad nema podataka. Loš empty state:

(prazna tablica bez ikakvog teksta)

Dobar empty state:

- Jasna poruka: „Nemate natjecatelja.”
- CTA (Call to Action): link ili gumb za dodavanje prvog zapisa
- Opcionalno: kratka uputa što sljedeće napraviti

Empty state se događa češće nego mislite - svaki novi korisnik vidi ga prvi put. To je prilika za dobar prvi dojam, ne prazna stranica.

Error stanja i retry

Nije svaka greška ista. Razlikujemo:

Tip	Primjer	Reakcija
Network error	Nema interneta	Retry gumb, provjeri konekciju
Server error (5xx)	Bug na backendu	Retry gumb, „Pokušajte kasnije”
Auth error (401)	Token istekao	Automatski refresh ili redirect
Client error (4xx)	Krivi podaci	Prikaži poruku, ne nudi retry

Retry ima smisla samo za tranzijentne greške (network, 5xx). Za 400/422/409 retry će dati isti rezultat - korisnik mora promijeniti input.

Debouncing: ne šalji request na svaki keystroke

Kad korisnik piše u search polje, ne želimo slati API request na svako slovo:

p → po → pow → powe → power = 5 requestova za jednu riječ

Debounce odgađa akciju dok korisnik ne prestane tipkati (npr. 300ms):

- 1 Korisnik piše p → timer kreće (300ms)
- 2 Piše o → timer se resetira
- 3 Piše w, e, r → timer se resetira svaki put
- 4 300ms bez tipkanja → šalje se jedan request za power

Za frontend filtriranje (computed) debounce nije potreban jer nema network poziva. Za API search endpoint je obavezan.

Composable: useDebouncedSearch

```
// src/composables/useDebouncedSearch.ts
import { ref, watch } from 'vue'

export function useDebouncedSearch(delay = 300) {
  const search = ref('')
  const debounced = ref('')
  let timer: ReturnType<typeof setTimeout>

  watch(search, (value) => {
    clearTimeout(timer)
    timer = setTimeout(() => { debounced.value = value }, delay)
  })

  return { search, debounced }
}

// u komponenti:
// const { search, debounced } = useDebouncedSearch()
// <input v-model="search" /> -> API poziv prati debounced
```

Composable je funkcija s prefiksom use koja kapsulira reaktivnu logiku - isto što i Pinia store, ali bez globalnog stanja. Svaka komponenta koja je pozove dobiva svoju instancu. Ovo je centralni Composition API pattern za dijeljenje logike.

Lista natjecatelja s UX stanjima

```
<script setup lang="ts">
import { onMounted } from 'vue'
import { useAuthStore } from '@/stores/auth'
import { useLifterStore } from '@/stores/lifters'

const auth = useAuthStore()
const store = useLifterStore()

onMounted(() => {
  store.fetchLifters(auth.user.club_id)
})
</script>

<template>
  <h1>Natjecatelji</h1>
  <div v-if="store.loading">Ucitavanje...</div>
  <div v-else-if="store.error" class="error">
    {{ store.error }}
    <button @click="store.fetchLifters(auth.user.club_id)">
      Pokušaj ponovo
    </button>
  </div>
  <div v-else-if="store.lifters.length === 0">
    Nemate natjecatelja. <RouterLink to="/lifters/new">
      Dodajte prvog.</RouterLink>
  </div>
  <table v-else>...</table>
</template>
```

Tablica podataka

```
<table v-else>
  <thead>
    <tr>
      <th>Ime</th>
      <th>Prezime</th>
      <th>Spol</th>
      <th>Datum rođenja</th>
      <th>Akcije</th>
    </tr>
  </thead>
  <tbody>
    <tr v-for="lifter in store.lifters" :key="lifter.id">
      <td>{{ lifter.first_name }}</td>
      <td>{{ lifter.last_name }}</td>
      <td>{{ lifter.gender }}</td>
      <td>{{ lifter.birth_date }}</td>
      <td>
        <RouterLink :to="`~/lifters/${lifter.id}/edit`">
          Uredi
        </RouterLink>
      </td>
    </tr>
  </tbody>
</table>
```

v-for iterira kroz array. :key je obavezan - Vue ga koristi za efikasno ažuriranje DOM-a.

Forma za dodavanje natjecatelja

```
<script setup lang="ts">
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import { AxiosError } from 'axios'
import api from '@api/client'
import { useAuthStore } from '@stores/auth'

const auth = useAuthStore()
const router = useRouter()
const form = ref({
  first_name: '', last_name: '',
  gender: 'M', birth_date: '',
})
const errors = ref<Record<string, string>>({})

async function submit() {
  errors.value = {}
  try {
    await api.post(
      `/clubs/${auth.user!.club_id}/lifters`, form.value
    )
    router.push('/lifters')
  } catch (e) {
    const err = e as AxiosError
    if (err.response?.status === 422) {
      errors.value = parseValidationErrors(err.response.data)
    }
  }
}
}
</script>
```

Forma: template s validacijom

```
<template>
  <h1>Novi natjecatelj</h1>
  <form @submit.prevent="submit">
    <div>
      <label>Ime</label>
      <input v-model="form.first_name" required />
      <span v-if="errors.first_name" class="error">
        {{ errors.first_name }}
      </span>
    </div>
    <div>
      <label>Spol</label>
      <select v-model="form.gender">
        <option value="M">Muski</option>
        <option value="F">Zenski</option>
      </select>
    </div>
    <div>
      <label>Datum rođenja</label>
      <input v-model="form.birth_date" type="date" required />
    </div>
    <button type="submit">Spremi</button>
  </form>
</template>
```

Edit forma: punjenje postojećim podacima

```
// src/views/LifterEditView.vue
<script setup lang="ts">
import { ref, onMounted, watch } from 'vue'
import { useRoute } from 'vue-router'
import api from '@api/client'
import type { Lifter } from '@types'

const route = useRoute()
const form = ref({ first_name: '', last_name: '',
                  gender: 'M', birth_date: '' })

async function load(id: number) {
  const { data } = await api.get<Lifter>(`/lifters/${id}`)
  form.value = { ...data }
}

onMounted(() => load(Number(route.params.id)))
watch(() => route.params.id,
      (id) => load(Number(id)))
</script>
```

onMounted pokriva prvi ulazak na rutu, a watch na route param slučaj kad router ponovno iskoristi istu komponentu za drugi ID - bez watcha forma ostane s podacima starog zapisa. Ovo je najčešći praktični watch u SPA aplikaciji.

Validacija: klijentska vs serverska

Klijentska (frontend)

- HTML atributi: required, min, maxlength
- Brzi feedback - bez mrežnog poziva
- Može se zaobići (DevTools)
- Za UX, ne za sigurnost

Serverska (backend)

- Pydantic schema validacija
- Vraća 422 s detaljima
- Ne može se zaobići
- Autoritet za podatke

Trebate obje. Klijentska daje brzi feedback, serverska jamči integritet. Frontend prikazuje serverske greške kad klijentska validacija nije dovoljna (npr. duplikat u bazi).

Form UX best practices

- ➊ **Inline validacija** - prikaži grešku ispod polja odmah kad korisnik napusti polje, ne tek na submit
- ➋ **Disable submit dok forma nije validna** - korisnik zna da nešto nedostaje
- ➌ **Clear error kad korisnik počne tipkati** - ne drži crveni tekst nakon što je korisnik počeo ispravljati
- ➍ **Focus na prvo polje s greškom** - nakon neuspjelog submита, premjesti fokus na problem
- ➎ **Serverske greške uz polje** - 422 greška za birth_date ide pored datumskog inputa, ne kao generička poruka

Prikaz serverskih grešaka u formi

Backend može vratiti dva tipa grešaka koje trebamo prikazati:

Status	Tip	Prikaz
422	Field-level validacija	Greška ispod odgovarajućeg polja (loc kaže koje)
409	Duplikat	Globalna poruka iznad forme: „Natjecatelj je već prijavljen”
400	Poslovno pravilo	Globalna poruka: „Rok za prijave je istekao”

Razlikujemo ih po status kodu. 422 ima detail array s lokacijama polja, 409 i 400 imaju message string. Frontend parsira response i prikazuje grešku na pravom mjestu.

Parsiranje 422 grešaka s backenda

```
// src/utils/errors.ts

// FastAPI 422 format:
// { "detail": [{ "loc": ["body", "first_name"],
// "msg": "field required", "type": "missing" }] }

interface ValidationDetail {
  loc: (string | number)[]
  msg: string
}

export function parseValidationErrors(
  data: { detail?: ValidationDetail[] } | unknown
): Record<string, string> {
  const errors: Record<string, string> = {}
  const detail = (data as { detail?: ValidationDetail[] })?.detail
  if (Array.isArray(detail)) {
    for (const err of detail) {
      // loc je ["body", "field_name"]
      const field = String(err.loc[err.loc.length - 1])
      errors[field] = err.msg
    }
  }
  return errors
}
```

Svaka greška ima loc (lokacija polja) i msg (poruka). Mapiramo ih na objekt {field: message} za prikaz u formi.

Registracije: prikaz sa status badgeom

```
<template>
  <table>
    <tr v-for="reg in registrations" :key="reg.id">
      <td>{{ reg.lifter_name }}</td>
      <td>{{ reg.category }}</td>
      <td>
        <span :class="statusClass(reg.status)">
          {{ reg.status }}
        </span>
      </td>
      <td>
        <button v-if="reg.status === 'active'"
          @click="withdraw(reg.id)">
          Odjavi
        </button>
      </td>
    </tr>
  </table>
</template>

<script setup lang="ts">
function statusClass(status: string): string {
  return status === 'active' ? 'badge-green' : 'badge-red'
}
</script>
```

Reusable komponente: kad izdvojiti

Izdvajamo komponentu kad se isti UI pattern ponavlja na 2+ mjesta:

Komponenta	Razlog
LoadingSpinner.vue	Koristi se u svakom viewu
ErrorMessage.vue	Isti pattern: poruka + retry
DataTable.vue	Tablica s headerom i redovima
FormField.vue	Label + input + error poruka
StatusBadge.vue	Obojeni badge za statuse

Ne pretjerujte s apstrakcijom. Ako se nešto koristi samo jednom, ne treba biti zasebna komponenta.

Kompozicija vs nasljeđivanje u UI-ju

Vue ne koristi nasljeđivanje komponenti. Umjesto toga koristi **kompoziciju**:

	Nasljeđivanje	Kompozicija
Pristup	<code>class AdminTable extends Table</code>	<code><DataTable></code> s propsima i slotovima
Fleksibilnost	Rigidna hijerarhija	Slobodna kombinacija
Reusability	Teško za različite kontekste	Lako - props i slots

Slots omogućuju parentu da definira **što** se prikazuje, a child komponenta definira **gdje** i **kako**. Ovo je isti princip kao „composition over inheritance” iz OOP-a.

Props dizajn: API vaše komponente

Propsi su **javni API** vaše komponente. Dizajnirajte ih s istom pažnjom kao REST endpoint:

- **Imenujte opisno:** `isLoading` umjesto `flag`, `errorMessage` umjesto `msg`
- **Definirajte tipove:** generički `defineProps<{...}>()` - TypeScript provjerava svaki prop na mjestu korištenja
- **Postavite defaulte:** `withDefaults()` za opcionalne propse
- **Koristite v-model za dvosmjerne propse:** umjesto `prop + emit` pattern ručno

Komponenta s 10+ propsova je znak da radi previše. Razmislite o razbijanju na manje komponente ili korištenju slotova.

Key prop u v-for: zašto je obavezan

Bez `:key`, Vue koristi „in-place patch” strategiju - ažurira postojeće DOM elemente umjesto da ih premjesti.

Problem: korisnik uređuje input u 2. redu tablice. Filtriranjem se 2. red promijeni - ali DOM element ostane isti. Input vrijednost pripada starom redu.

Key	Ponašanje
<code>:key="item.id"</code>	Vue prati identitet elementa - premješta DOM node kad se lista presloži
<code>:key="index"</code>	Vue tretira poziciju kao identitet - bugovi pri sortiranju/filtriranju
Bez keya	Warning u konzoli, nepredvidljivo ponašanje

Koristite jedinstven, stabilan identifikator (ID iz baze) kao key. Nikad index.

Emit pattern: inverzija kontrole

Child komponenta ne odlučuje što se događa kad korisnik klikne gumb - samo javlja parentu.

Child: `emit('delete', itemId)` → Parent: `handleDelete(itemId)`

- Child ne zna koristi li se u admin viewu ili club viewu
- Parent odlučuje: prikaži modal? Odmah obriši? Zatraži razlog?
- Ista child komponenta se koristi u različitim kontekstima s različitim ponašanjem

Ovo je **inverzija kontrole** - isti princip kao dependency injection na backendu. Komponenta prima ponašanje izvana umjesto da ga hardkodira.

Reusable FormField komponenta

```
<!-- src/components/FormField.vue -->
<script setup lang="ts">
defineProps<{
  label: string
  error?: string
  modelValue: string | number
}>()
defineEmits<{
  'update:modelValue': [value: string]
}>()
</script>

<template>
  <div class="form-field">
    <label>{{ label }}</label>
    <input :value="modelValue"
      @input="$emit('update:modelValue',
        ($event.target as HTMLInputElement).value)"
    />
    <span v-if="error" class="error">{{ error }}</span>
  </div>
</template>

<!-- Koristenje: -->
<!-- <FormField v-model="form.first_name" label="Ime"
  :error="errors.first_name" /> -->
```

Commitovi za Predavanje 9

- `feat(web): add lifters list view with loading/error/empty states`
- `feat(web): add lifter create/edit form with validation`
- `feat(web): add competitions list and registration form`
- `feat(web): add my registrations view with status badges`
- `refactor(web): extract reusable form and table components`

Na satu: gradimo CRUD viewove, testiramo forme, promatramo Network tab u DevToolsima.

Što ide sljedeće (Predavanje 10)

- Admin viewovi: tablice s filterima i paginacijom
- Permission-aware UI - prikaz/sakrivanje ovisno o roli i fazi
- Modali za potvrdu akcija
- CSV export

Pitanja?